



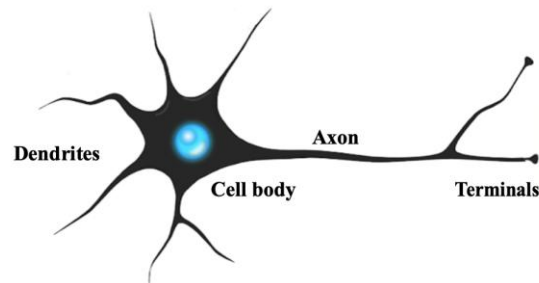
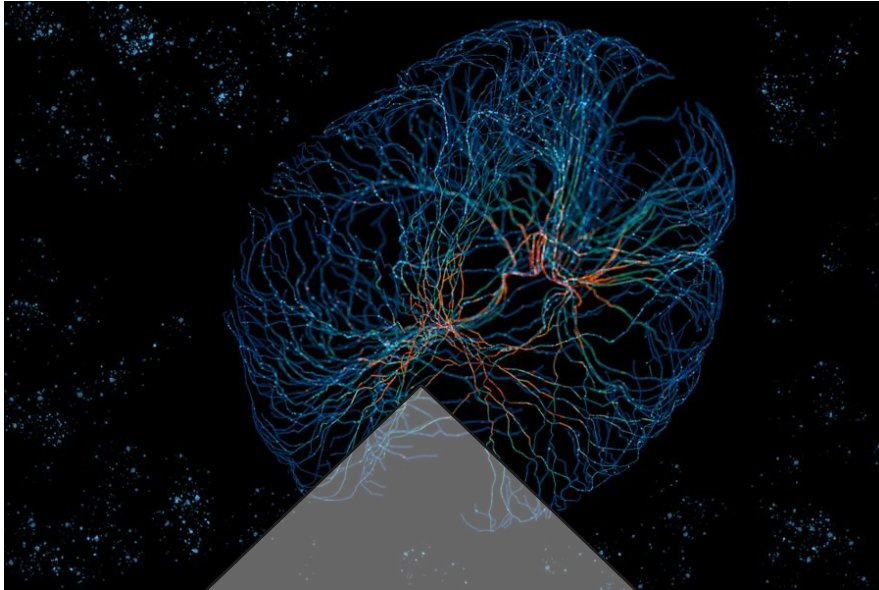
APPLIED DEEP LEARNING

03 – ARTIFICIAL NEURAL NETWORKS (BASICS) – I

PROF. DR. LARS ACKERMANN-IGL

A (VAGUE) INSPIRATION

Real Neural Networks



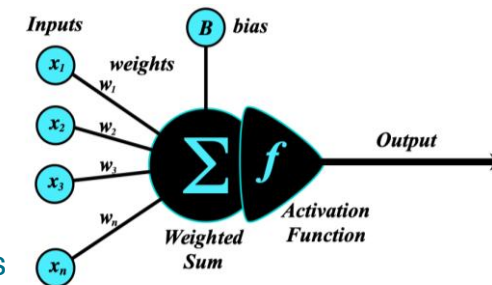
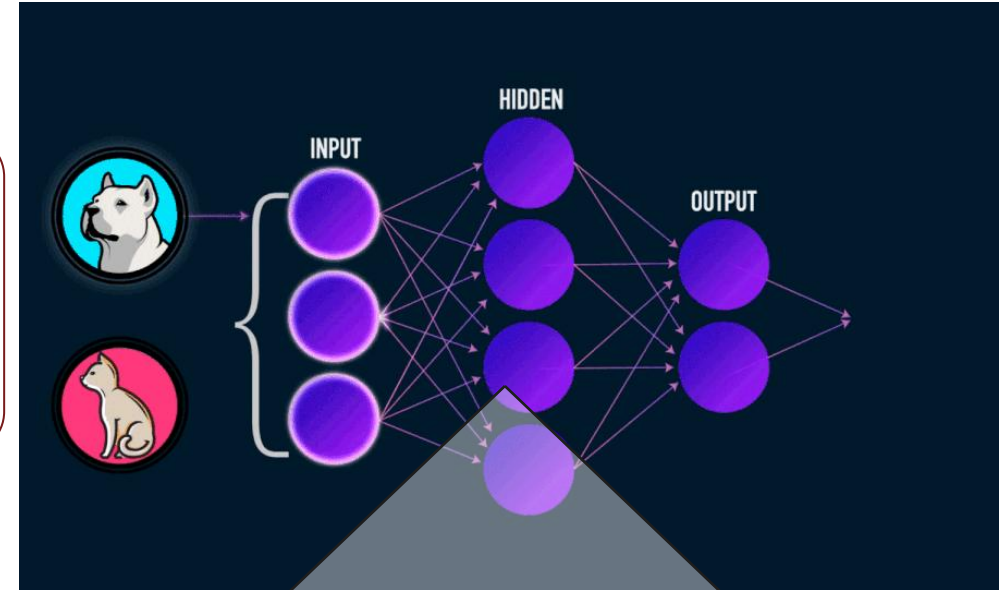
Real Neurons

So ... are
Artificial
Neural
Networks
similar to
biological?



Stay tuned!

Artificial Neural Networks



Artificial Neurons

RECAP: ADVANTAGES OF DEEP LEARNING – USING ARTIFICIAL NEURAL NETWORKS

We discussed the value already from the feature engineering perspective ...

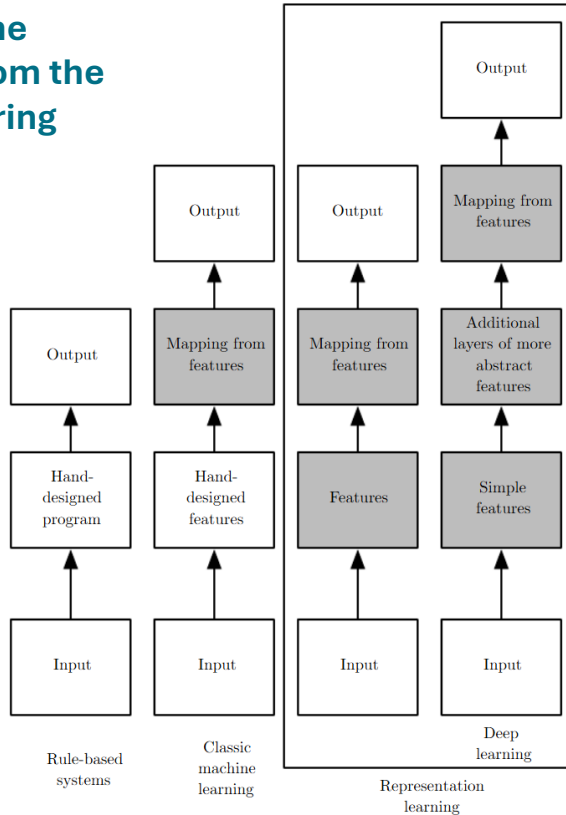


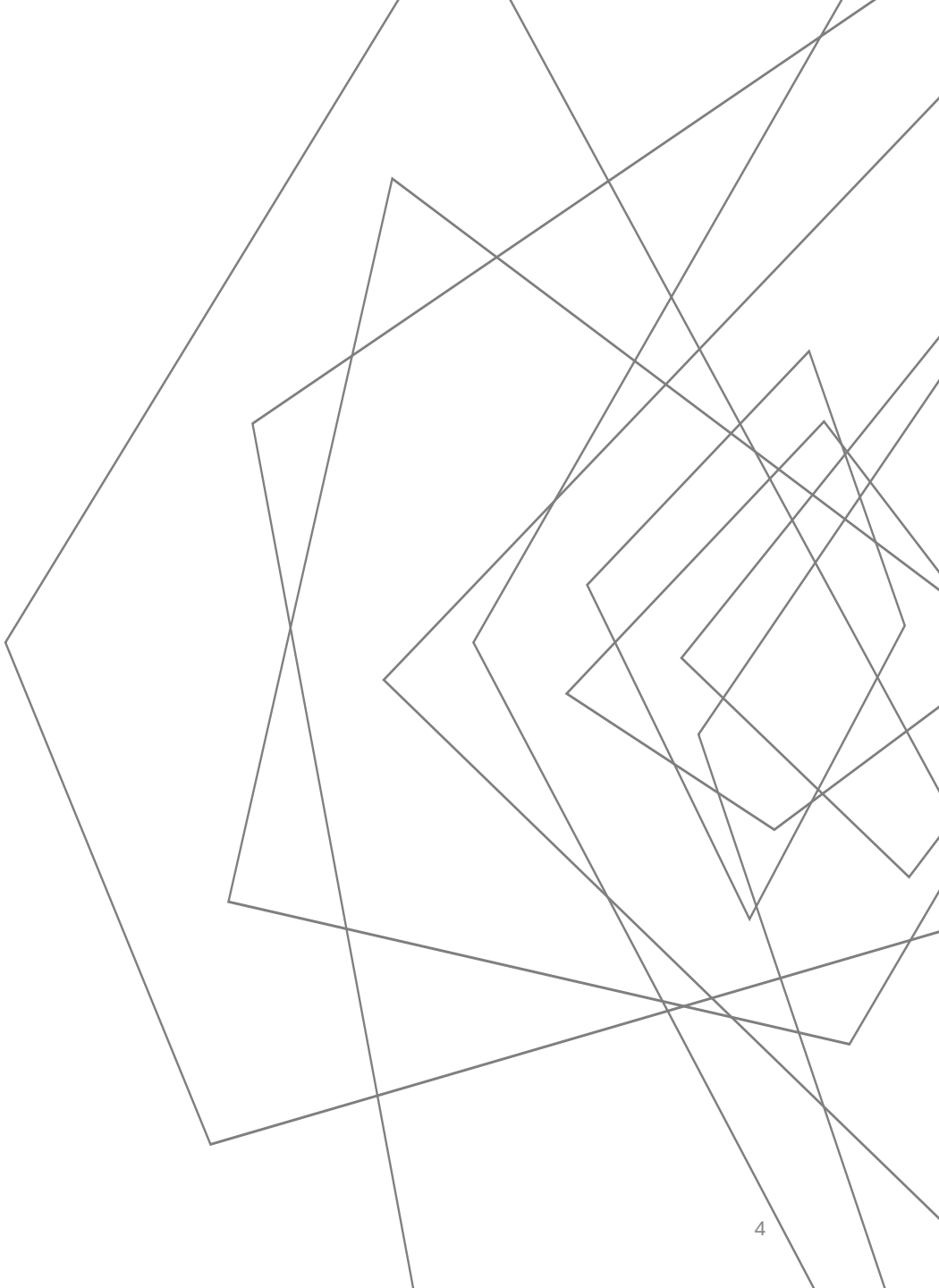
Figure 1.5: Flowcharts showing how the different parts of an AI system relate to each other within different AI disciplines. Shaded boxes indicate components that are able to learn from data.

... but there are more advantages:

- **Pattern Recognition:** ANNs excel at identifying patterns and relationships in large datasets, making them ideal for tasks like image and speech recognition.
- **Non-linearity:** They can model complex, non-linear relationships, which traditional linear models may struggle to capture.
- **Adaptability:** ANNs can learn from new data, enabling them to adapt to changes in the underlying data distribution over time.
- **Generalization:** With proper training, ANNs can generalize from the training data to unseen data, making them effective for prediction tasks.
- **Parallel Processing:** ANNs can process multiple inputs simultaneously, allowing for faster computation and performance in real-time applications.
- **Robustness:** They can handle noisy and incomplete data better than many traditional algorithms, making them suitable for real-world applications.
- **Versatility:** They can be applied across various domains, including healthcare, finance, natural language processing, and robotics, among others.
- **Scalability:** ANNs can be scaled up to handle large datasets and complex models, particularly with advancements in hardware and cloud computing.
- **Transfer Learning:** Pre-trained models can be fine-tuned on new tasks, saving time and resources in training.

Written by an AI ... see

<https://www.quora.com/What-are-the-benefits-of-artificial-neural-networks>

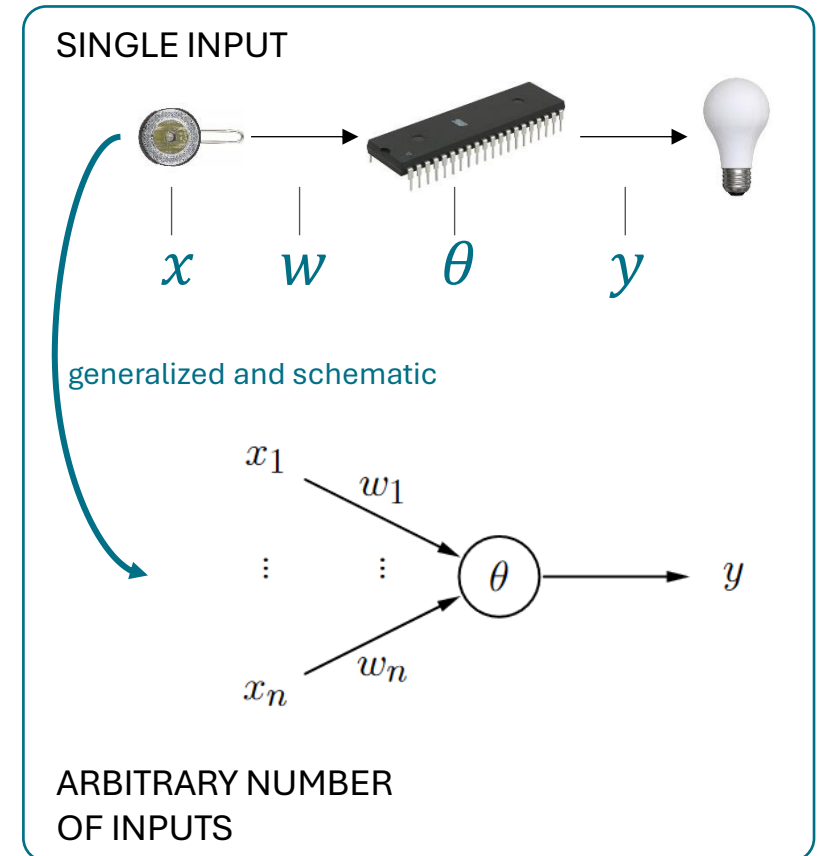


THRESHOLD LOGIC UNITS (TLUs)

- Definition and geometric interpretation
- Learning parameters of Threshold Logic Units (aka Training)
 - Why manual training is not feasible
 - How automated training works
- Limitations of TLUs
 - Linear separability
 - Solution: networks of TLUs

THRESHOLD LOGIC UNITS (TLUs) – OVERVIEW

- **Relevance:** Oldest form of a neural network¹
- **Basic idea:** A simple unit produces one output value depending on one input value e.g., a controller turns the light on/off depending on a light sensor
- **Operating principle (informally):** The controller defines a threshold θ that must be reached, before it “sends a 1” to the light bulb, turning it on
- **Generalized structure:**
 - inputs x
 - weights w (also: “parameters”),
 - threshold θ (“theta”)
 - output y
- **Questions:**
 - 1) How is the output calculated?
 - 2) Why are there weights?



¹ MCCULLOCH, Warren S.; PITTS, Walter. A logical calculus of the ideas immanent in nervous activity. *The bulletin of mathematical biophysics*, 1943, 5. Jg., S. 115-133.

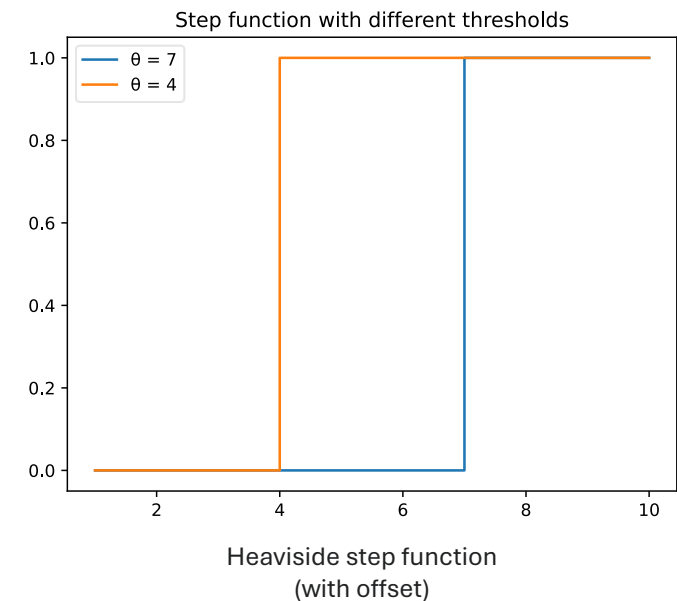
THRESHOLD LOGIC UNITS (TLUs)

- **Question 1: How is the output calculated?**

$$y = f(xw) = \begin{cases} 1, & \text{if } xw \geq \theta \\ 0, & \text{otherwise} \end{cases}, \text{ with } xw = \sum_{i=1}^n w_i x_i \longrightarrow$$

- **Question 2: Why are there weights?**

- Each input x_i is weighted with w_i to indicate its importance for the output (in terms of distinctiveness)
- Example 1 (stop sign detection for self-driving cars): octagon shape is more important than red color
- Example 2 (credit card fraud detection): transaction amount vs. location of transaction ... but what is more important?



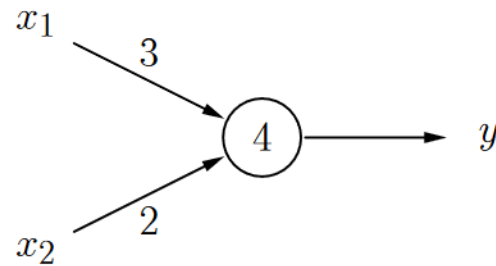
EXAMPLES AND THEIR GEOMETRIC INTERPRETATION

TLU

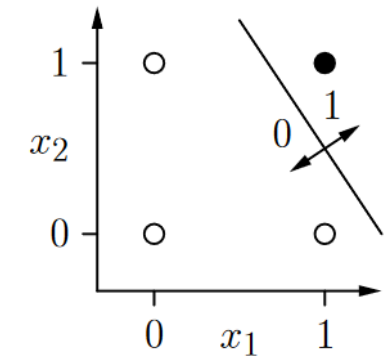
Truth table

Geometric Interpretation

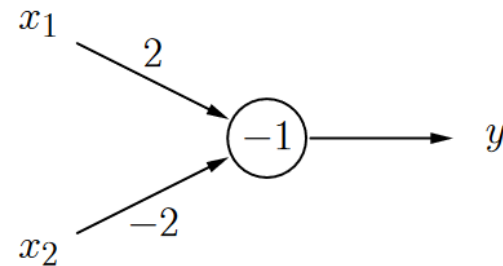
- **Conjunction** $x_1 \wedge x_2$



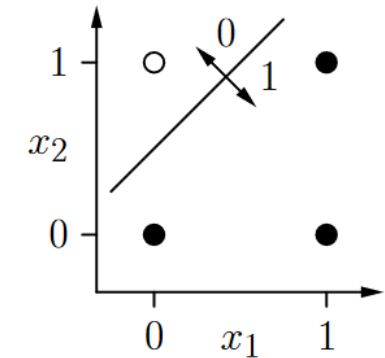
x_1	x_2	$3x_1 + 2x_2$	y
0	0	0	0
1	0	3	0
0	1	2	0
1	1	5	1



- **Implication** $x_2 \rightarrow x_1$

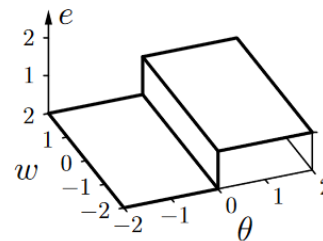
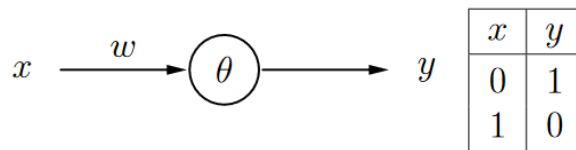


x_1	x_2	$2x_1 - 2x_2$	y
0	0	0	1
1	0	2	1
0	1	-2	0
1	1	0	1

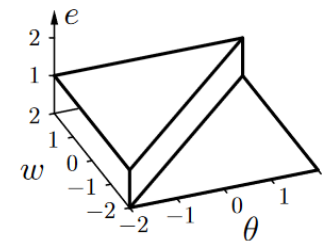


LEARNING PARAMETERS OF THRESHOLD LOGIC UNITS – 1

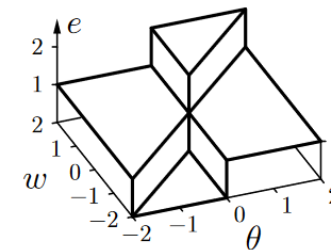
- Until now: TLUs constructed **manually** regarding weights and threshold (e.g., done by looking at geometric representation)
 - Therefore, this approach is only possible for a maximum of three inputs
- More convenient: Letting an **automated** system choose weights and threshold based on given data
 1. Start with random weights and threshold assigned to a TLU
 2. With a given data set calculate the **error e** that the TLU produces
 - For the time being: an error is the (absolute) difference between expected and calculated value
 - e is dependent on our set of weights w and the threshold θ
 3. Tweak weights and threshold to make this error smaller
 4. Repeat from step 2 until the error is sufficiently small (or vanishes completely)
- Example for a TLU representing a negation of a single input ($\neg x$):



Error e_0 for $x = 0$



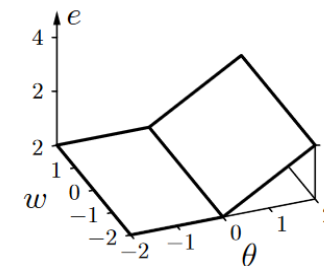
Error e_1 for $x = 1$



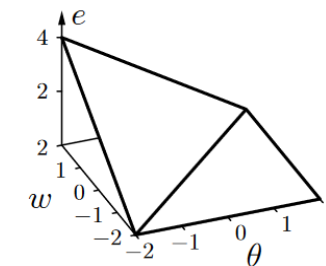
Combined Error
 $e = e_0 + e_1$

LEARNING PARAMETERS OF THRESHOLD LOGIC UNITS – 2

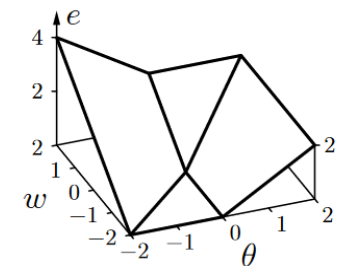
- The error surface (from slide before) doesn't lend itself to being used in optimization, since it only consists of **plateaus**
→ Given any point on the surface, we never know in which direction to adjust our w and θ
- **Solution:** Multiply the error by the weighted sum's distance to the threshold
 - i. considers, how far the weighted sum is from the threshold and
 - ii. now a direction can be calculated that reduces the error (only as schematic here)



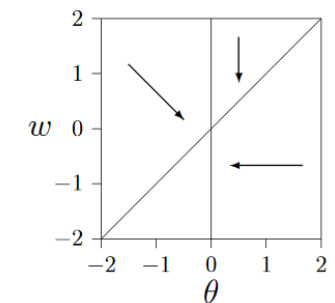
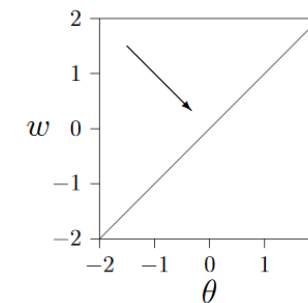
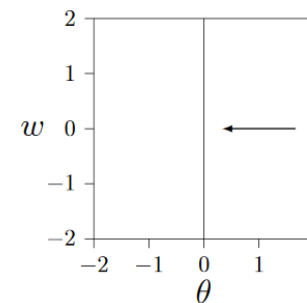
Error e_0 for $x = 0$



Error e_1 for $x = 1$



Combined Error
 $e = e_0 + e_1$

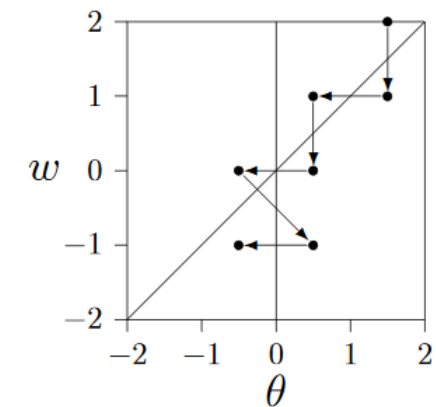
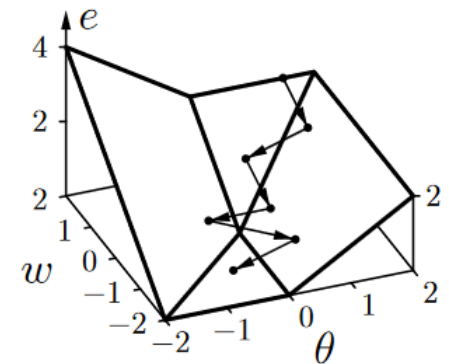


THE DELTA RULE (FOR TLUs)

- Following this direction (of decreasing error value) and the procedure from earlier, weights and threshold can be adjusted to reduce the error
- More formally let $\mathbf{x} = (x_1, \dots, x_n)$ be an input vector for which we know the desired output $\mathbf{o}$. Given $\mathbf{x}$ our (randomly initialized) TLU produces the output $\mathbf{y}$. We can then adjust $\mathbf{w}$ and θ

$$\begin{aligned} \theta^{(new)} &= \theta^{(old)} + \Delta\theta, & \text{with } \Delta\theta &= -\eta(o - y) \text{ and} \\ \forall i \in \{1, \dots, n\} \quad w_i^{(new)} &= w_i^{(old)} + \Delta w_i, & \text{with } \Delta w_i &= \eta(o - y)x_i \end{aligned}$$

- η (“eta”) is called the **learning rate** and controls the magnitude of change to $\mathbf{w}$ and θ
- This procedure is called **delta rule** or **Widrow–Hoff Procedure**



TRANSFORMING THRESHOLDS TO WEIGHTS

- Until now, we always had to distinct between two cases when adjusting parameters

- Threshold θ

- Weights $\mathbf{w}$

$$\sum_{i=1}^n w_i x_i \geq \theta$$

- Having only one case would simplify the training algorithm
- Idea:** Think of the threshold as an additional input with its own weight by transforming the equation from above

$$\sum_{i=1}^n w_i x_i - \theta \geq 0, \text{ with } -\theta = w_0 x_0$$

- θ can then be implemented by a “special” weight $\mathbf{w}_0$ and constant input $x_0 = 1$, while still serving the same purpose



THE DELTA RULE (GENERALIZED)

CHANGED OUTPUT CALCULATION

$$y = f(xw) = \begin{cases} 1, & \text{if } xw \geq \theta \\ 0, & \text{otherwise} \end{cases}, \text{ with } xw = \sum_{i=1}^n w_i x_i$$

↓

$$y = f(xw) = \begin{cases} 1, & \text{if } xw \geq 0 \\ 0, & \text{otherwise} \end{cases}, \text{ with } xw = \sum_{i=0}^n w_i x_i$$

OLD DELTA RULE

$$\forall i \in \{1, \dots, n\} \quad \begin{aligned} \theta^{(new)} &= \theta^{(old)} + \Delta\theta, & \text{with } \Delta\theta &= -\eta(o - y) \text{ and} \\ w_i^{(new)} &= w_i^{(old)} + \Delta w_i, & \text{with } \Delta w_i &= \eta(o - y)x_i \end{aligned}$$

NEW DELTA RULE

$$\forall i \in \{0, \dots, n\} \quad w_i^{(new)} = w_i^{(old)} + \Delta w_i, \quad \text{with } \Delta w_i = \eta(o - y)x_i$$

APPLYING THE DELTA RULE

$$\forall i \in \{0, \dots, n\} \quad w_i^{(new)} = w_i^{(old)} + \Delta w_i, \quad \text{with } \Delta w_i = \eta(o - y)x_i$$

TRAINING PROCEDURE:

Training data: $S = \{(x_1 = 1, x_2 = 1, y = 1)\}$

For reasons of simplicity, we set $\eta = 1$. Note that you won't do this in practice!

Step 1: Calculate the output for the training sample S_1

$$y = f(-5 * 1 + 2 * 1 + 1 * 1) = f(-2) = 0$$

Step 2: Calculate the Δw_i for training sample S_1

$$\Delta w_0 = 1(1 - 0)1 = 1$$

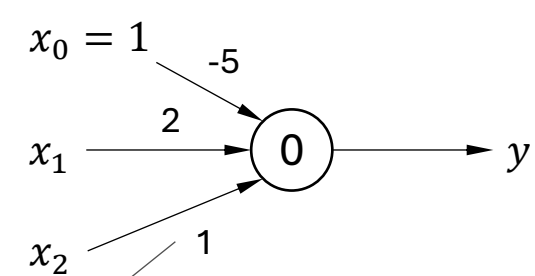
$$\Delta w_1 = 1(1 - 0)1 = 1$$

$$\Delta w_2 = 1(1 - 0)1 = 1$$

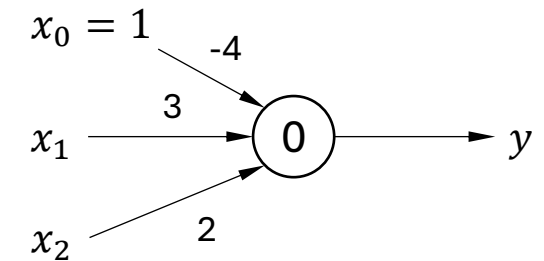
Step 3: Adjust weights

$$w_0^{(new)} = -5 + 1 = -4, w_1^{(new)} = 2 + 1 = 3, w_2^{(new)} = 1 + 1 = 2$$

INITIAL MODEL



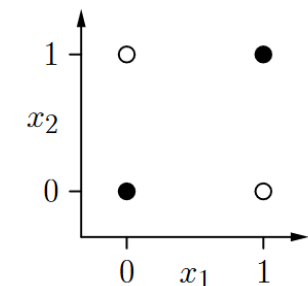
TRAINED MODEL



LIMITATION OF THRESHOLD LOGIC UNITS

- A single TLU can only calculate **linear separable functions**
- **Linear Separability**
 - Two sets of points in a Euclidean space are called linearly separable, iff there exists at least one point, line, plane or hyperplane (depending on the dimension of the Euclidean space), such that all points of the one set lie on one side and all points of the other set lie on the other side of this point, line, plane or hyperplane (or on it). That is, the point sets can be separated by a linear decision function.
- Examples for linearly separable functions
 - Conjunction, disjunction, implication, NAND, NOR, ...
- Examples for not linearly separable functions
 - **Bi-implication:**
$$x_1 \leftrightarrow x_2 \equiv (x_1 \rightarrow x_2) \wedge (x_2 \rightarrow x_1)$$
 - XOR, ...

x_1	x_2	y
0	0	1
1	0	0
0	1	0
1	1	1





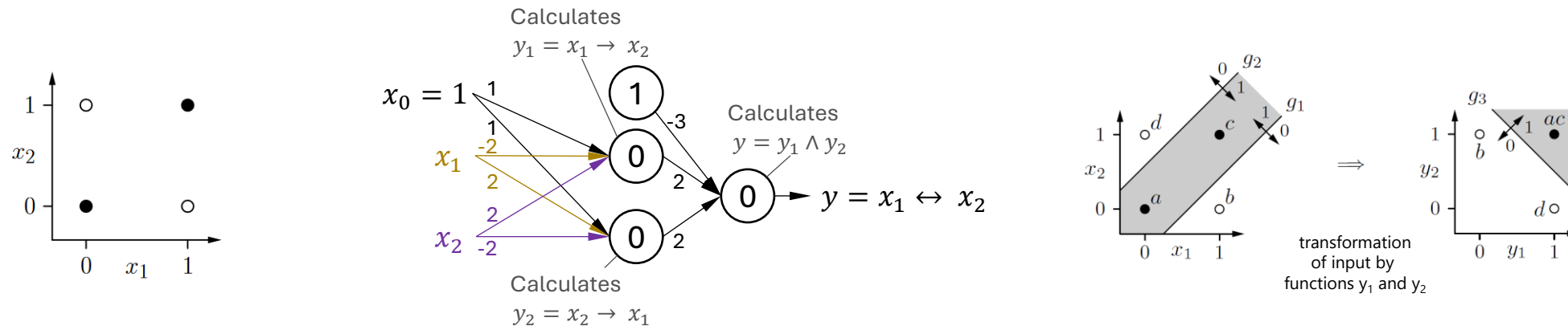
FUNDAMENTALS OF NEURAL NETWORKS

- Networks of TLUs and Inference
- Artificial neural networks
 - Artificial neurons
 - Input, activation, and output function
- Common activation functions
- Components of neural networks
- Network Topologies

SIMPLE EXTENSION: NETWORKS OF TLUs

- Single TLUs can only calculate **linear separable functions**, to calculate arbitrary Boolean functions, use a **network of TLUs**

- Example: bi-implication $x_1 \leftrightarrow x_2 \equiv (x_1 \rightarrow x_2) \wedge (x_2 \rightarrow x_1)$



- Total number (2^{2^n}) and number of linearly separable Boolean functions

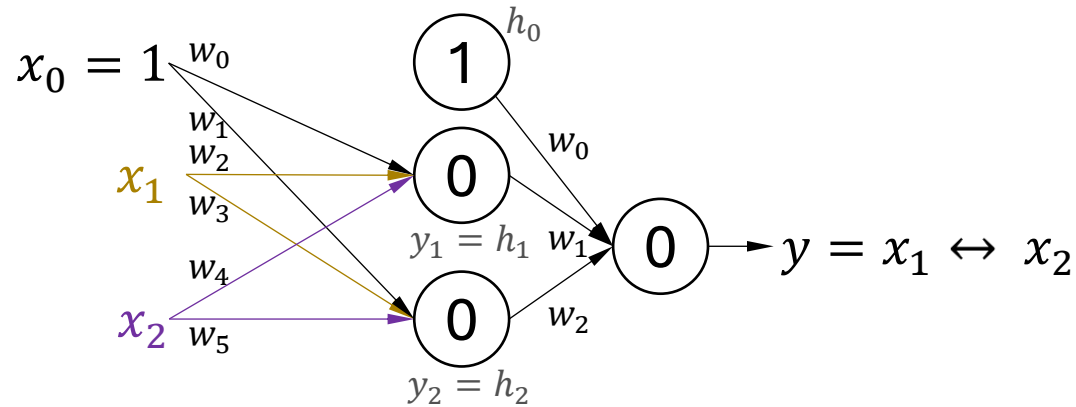
→ networks of TLUs are needed

inputs	Boolean functions	linearly separable functions
1	4	4
2	16	14
3	256	104
4	65,536	1,882
5	4,294,967,296	94,572
6	18,446,744,073,709,551,616	15,028,134
n	$2^{(2^n)}$	no general formula known

→ Boolean functions up to 2 input variables (all functions but XOR and XNOR are linearly separable)

$f_0 = 0$	False
$f_1 = AB$	AND
$f_2 = A\bar{B}$	AND (not B)
$f_3 = A$	Identity
$f_4 = \bar{A}B$	AND (not A)
$f_5 = B$	Identity
$f_6 = \bar{A}B + A\bar{B}$	Exclusive OR
$f_7 = A + B$	OR
$f_8 = \bar{A}\bar{B} = \bar{A+B}$	NOR
$f_9 = \bar{A}B + A\bar{B}$	Equality
$f_{10} = \bar{B}$	NOT
$f_{11} = A + \bar{B}$	OR (not B)
$f_{12} = \bar{A}$	NOT
$f_{13} = \bar{A} + B$	OR (not A)
$f_{14} = \bar{A} + \bar{B} = \overline{AB}$	NAND
$f_{15} = 1$	True

INFERENCE WITH NETWORKS OF TLUs



$$f(x) = \begin{cases} 1, & \text{if } \mathbf{xw} \geq 0 \\ 0, & \text{otherwise} \end{cases}, \text{ with } \mathbf{xw} = \sum_{i=0}^n w_i x_i$$

Thus: $y = f(f(x))$ as it is a two-layer network

Step 1:

$$y_1 = f\left(\frac{w_0 x_0}{1} - \frac{w_1 x_1}{2} + \frac{w_2 x_2}{2}\right) = f(-1) = 0$$

$$y_2 = f\left(\frac{w_0 x_0}{1} + \frac{w_1 x_1}{2} - \frac{w_2 x_2}{2}\right) = f(3) = 1$$

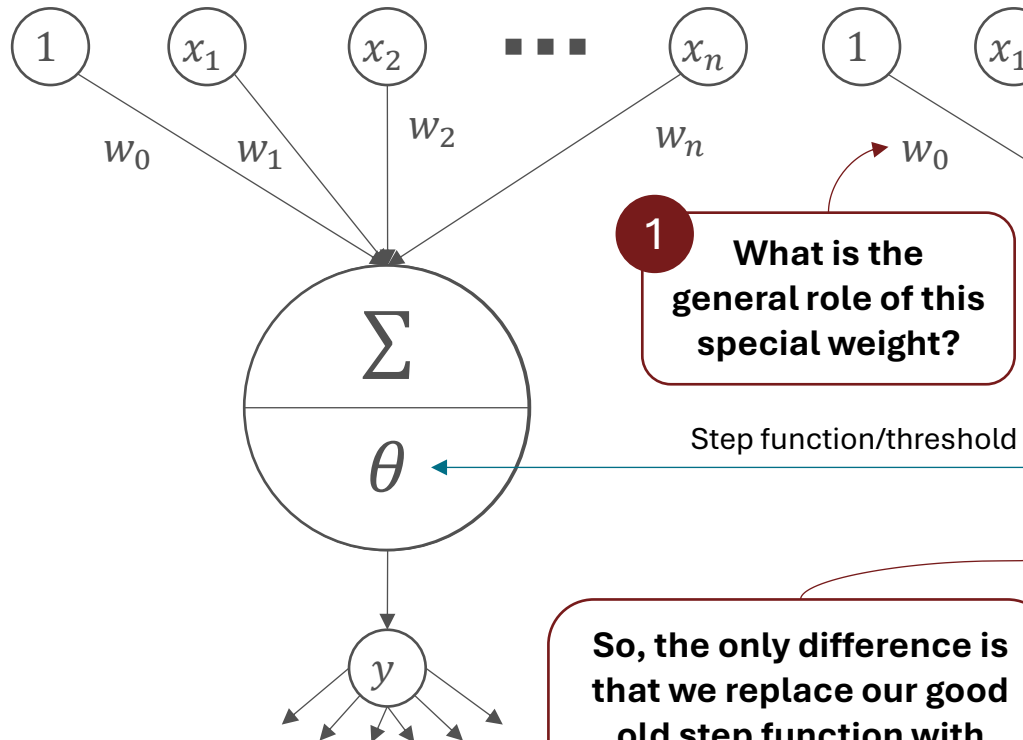
Step 2:

$$y = f\left(\frac{w_0 h_0}{-3} + \frac{w_1 h_1}{2} + \frac{w_2 h_2}{2}\right) = f(-1) = 0$$

How to train the hidden weights?

FROM TLUs TO NEURONS

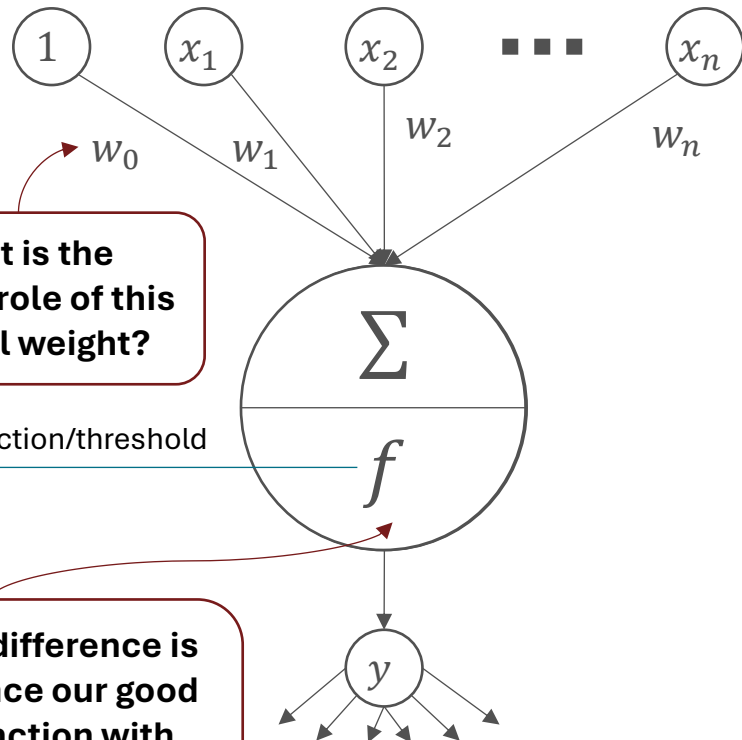
TLUs



1 What is the general role of this special weight?

2 So, the only difference is that we replace our good old step function with something new! But with what exactly?

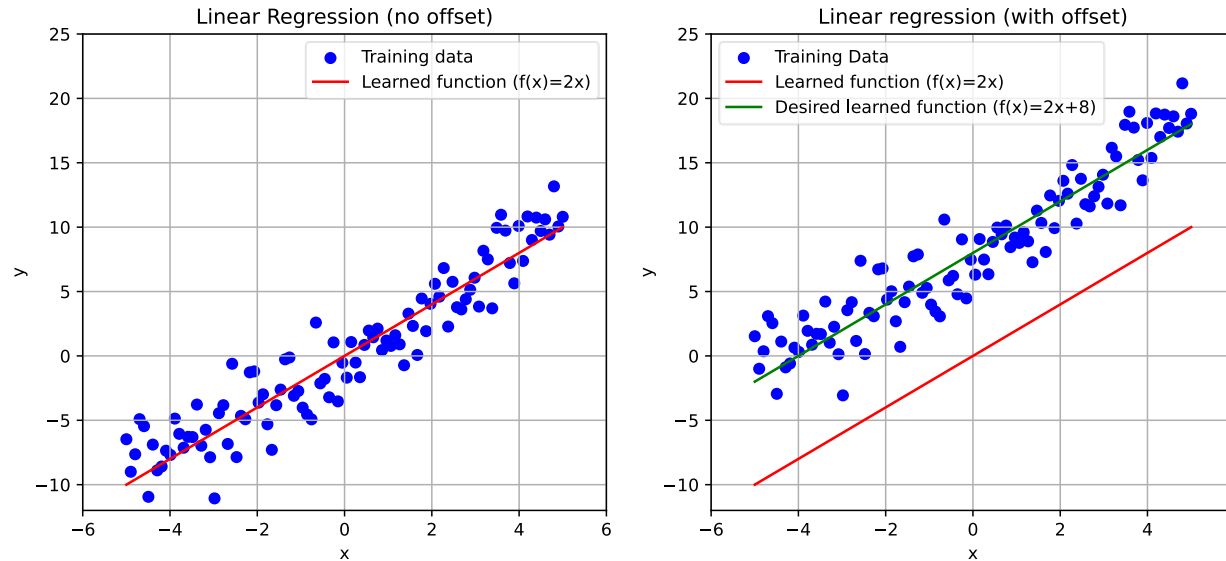
Neurons



Definitions

- $x_1 \dots x_n$ = input signals with $x_i \in \mathbb{R}$
- $w_1 \dots w_n$ = input weights
- $\sum w_i x_i$ = weighted sum as input
- f = **activation function**
- y = $f(w, x) = f(w \cdot x) = f(\sum_{i=0} w_i x_i)$
= output signal

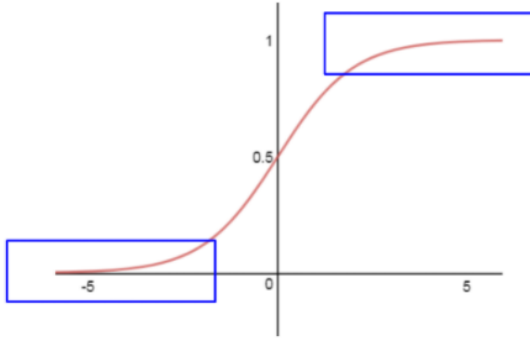
1 FUNCTION APPROXIMATION AND BIAS



- **Let's consider the following problem :** We want to approximate a linear function, e.g., in a regression scenario (see figure above).
- **What we have so far:** We approximate functions with $y = f(w, x) = f(wx) = f(\sum_{i=0} w_i x_i) = f(\sum_{i=1} w_i x_i + w_0)$,
- **The role of the special weight:** Enables us to approximate functions that do NOT intercept the axes at the coordinate origin (0,0), i.e., it shifts the activation function!
- Name of the construction: **Bias** $\textcircled{1} \xrightarrow{w_0}$
- Consider the general form of linear equations: $f(x) = ax + c \rightarrow$ **weights = slope, bias = intercept!**

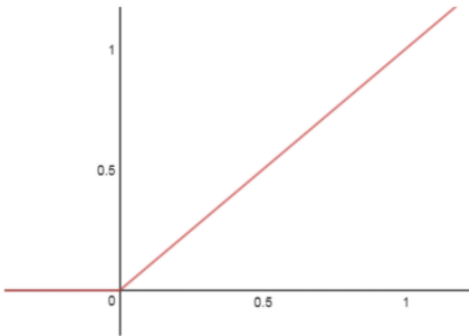
2 COMMONLY USED ACTIVATION FUNCTIONS

Sigmoid function: $f(x) = \frac{1}{1 + e^{-x}}$



- Gradient problem
- Not centered around 0
- **complex** in computation

Rectified Linear Unit (ReLU): $R(x) = \begin{cases} x & \text{if } x \geq 0 \\ 0 & \text{otherwise} \end{cases}$

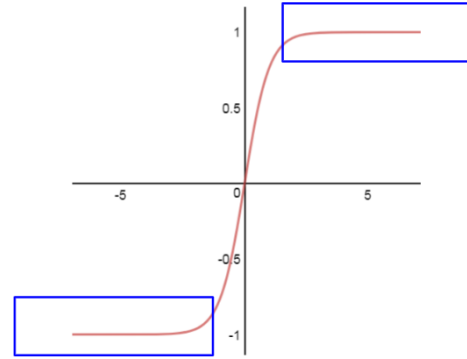


+ very easy to compute (in comparison to sig or tanh)

+ Gradient problem solved

o it can happen, that there are neurons in the NN which will never be activated (“dead neurons“)

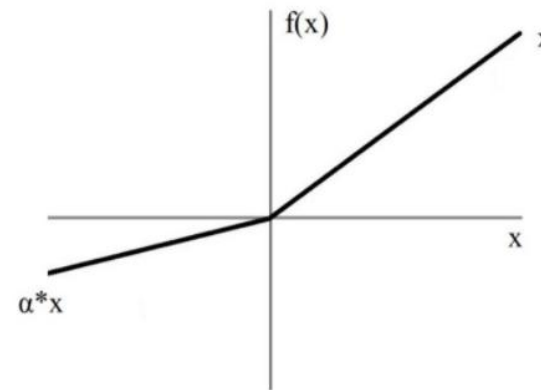
Tangens hyperbolicus (tanh) function: $f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$



+ same properties as sigmoid, but in addition **centered around the value 0**

- Gradient problem still exists
- even **more complex** than sig

Leaky ReLU: $R(x) = \begin{cases} x & \text{if } x \geq 0 \\ \alpha x & \text{otherwise} \end{cases}$

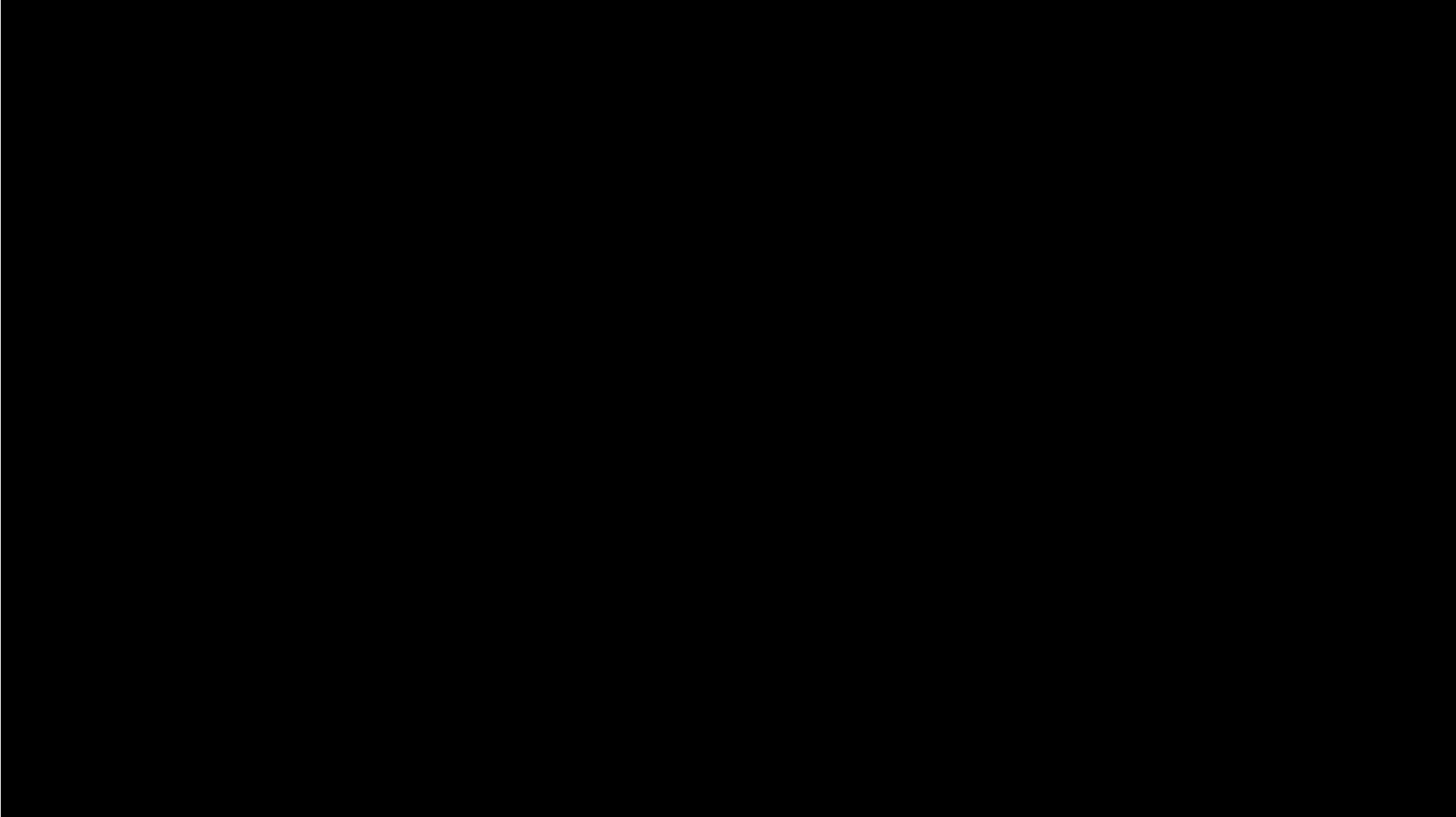


+ solves the “problem“ of “dead neurons“

SIDENOTE

In practice, it is not necessarily bad if there are some dead neurons!

SO ... ARE THOSE ARTIFICIAL NEURONS SIMILAR TO BIOLOGICAL NEURONS NOW?

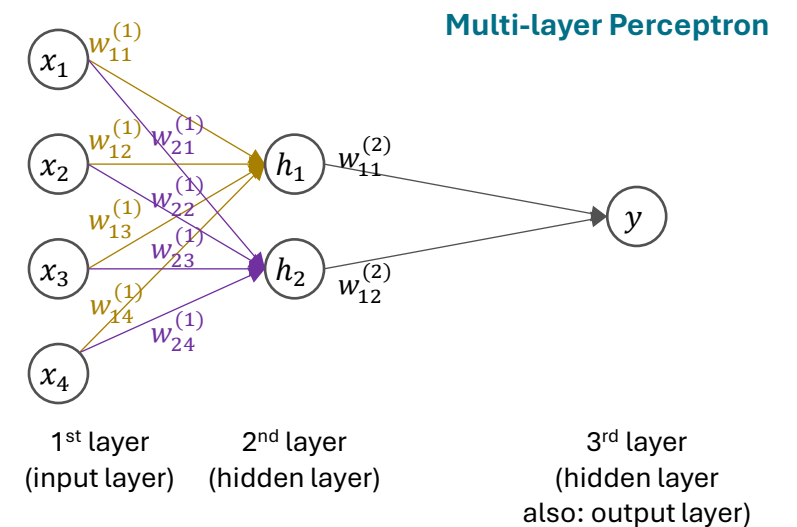
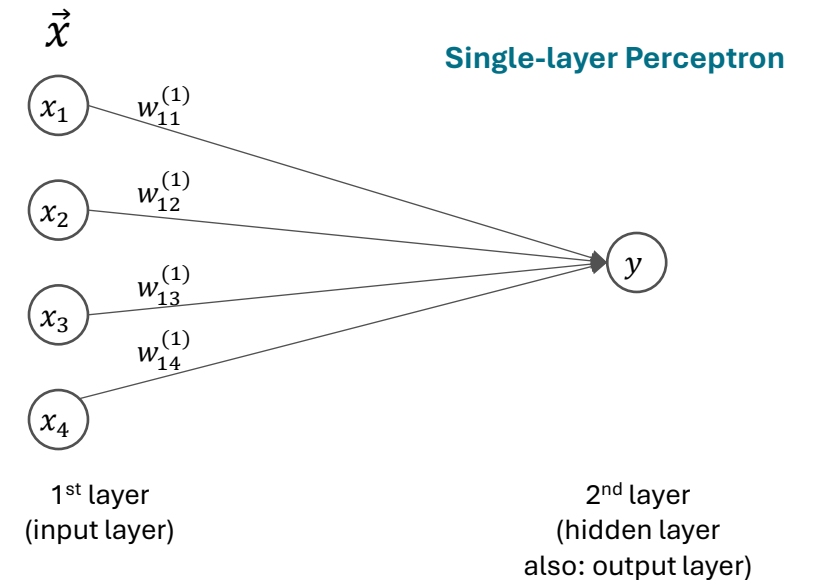


One of many perspectives (here the functional one) on reasons, why this is not the case!

ARTIFICIAL NEURAL NETWORKS (ANNs)

- **Artificial neural network (Definitions):**

- Directed graph $G = (U, C)$
- U is a finite set of **nodes** called units (the **neurons**)
- $C \subseteq U \times U$ is a finite set of **edges** called **connections**
- $c \in C$; each connection is directed from node u to node v and has a corresponding weight
- The **weight** of the connection between the u^{th} neuron of layer l and the v^{th} neuron of layer $l + 1$ is called $w_{uv}^{(l)}$ (Note: index u belongs to layer $l + 1$ and v to layer l as defined in literature)
- The **value** y_u of a neuron u is its output
- Neurons are grouped to **layers**; the output of a layer is a vector consisting of the values of all its neurons
- Dimensionality of these vectors/layers determines the model's **width**
- The number of hidden layers determines the model's **depth**

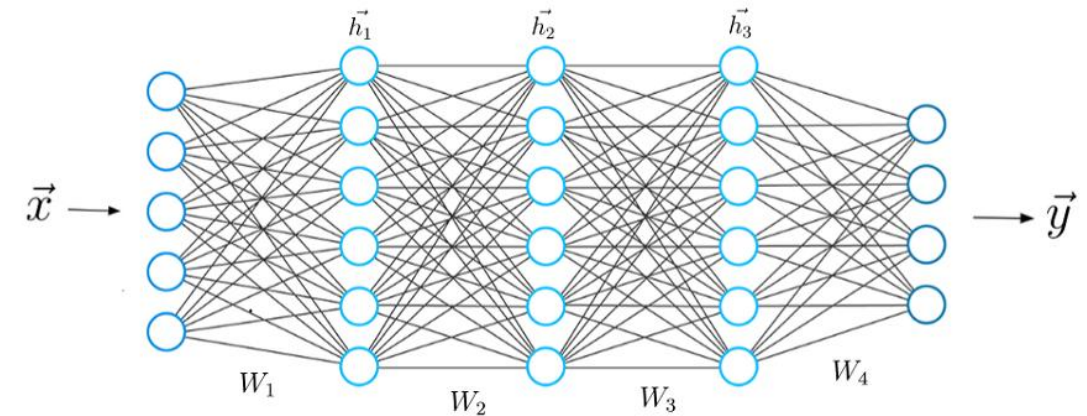


INFERENCE WITH ANNs – FORWARD PROPAGATION

- **Forward Propagation:** How inference with neuronal network works

- Input vector x
- Output vector y
- Weight matrix for each layer: W_1, W_2, W_3, W_4

- Calculate:
$$\begin{aligned}\vec{h}_1 &= f(\vec{x} \cdot W_1) \\ \vec{h}_2 &= f(\vec{h}_1 \cdot W_2) \\ \vec{h}_3 &= f(\vec{h}_2 \cdot W_3) \\ \vec{y} &= f(\vec{h}_3 \cdot W_4)\end{aligned}$$

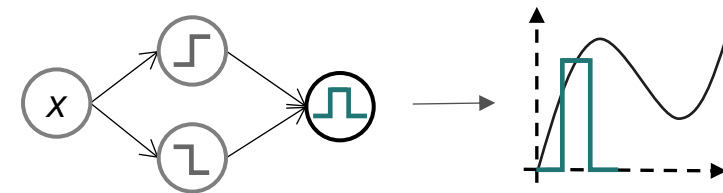


- Note: Implementations typically use this matrix representation since computations are way more efficient

- Summarizing the effect of a neuronal network

$$\vec{y} = f(xw) = f^{(n)}(\dots f^{(2)}(f^{(1)}(\vec{x} \cdot W_1) \cdot W_2) \dots \cdot W_n)$$

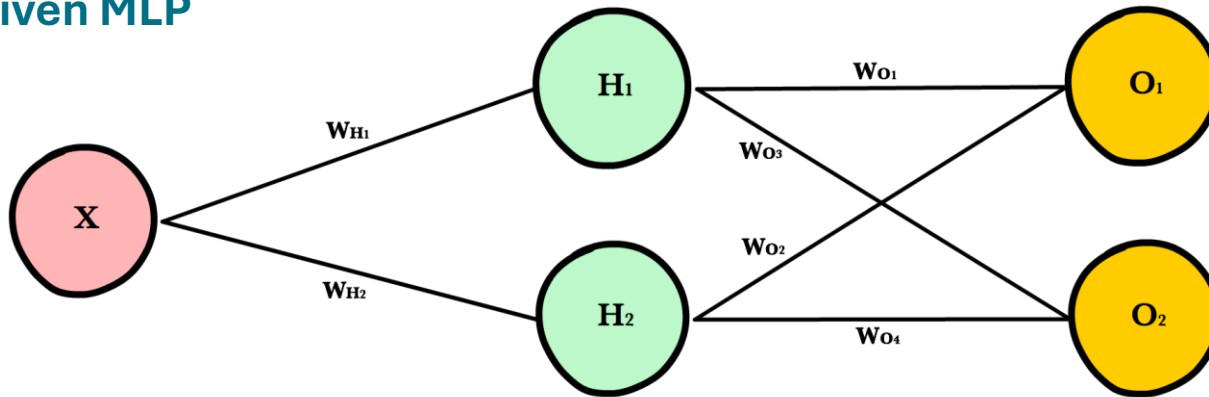
- **Remember:** Neural networks with a single hidden layer are already universal function approximators.



- Increasing number of neurons in hidden layer improves approximation

EXAMPLE: FORWARDPASS IN AN MLP

Given MLP



Symbols

Z_H

... weighted input for hidden layer

B_H

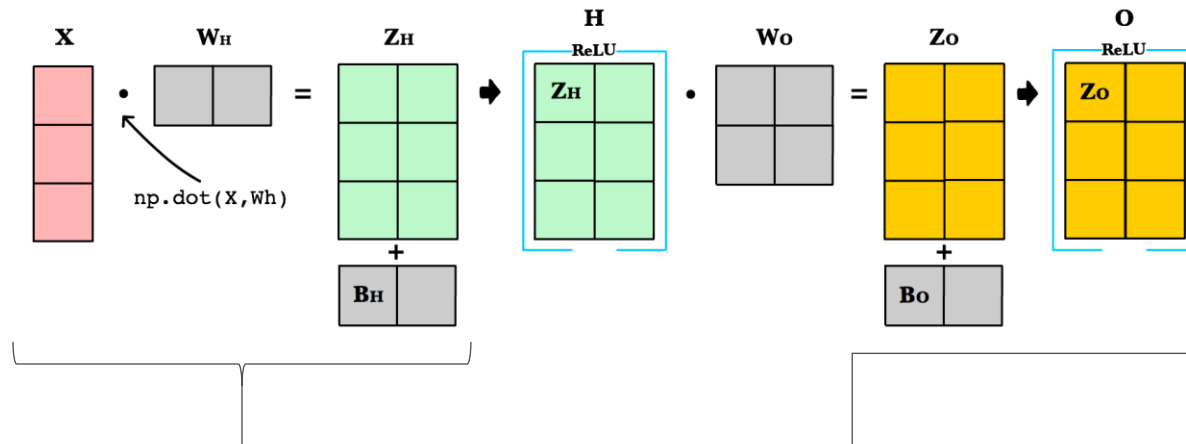
... bias vector for hidden layer

H

... hidden layer output matrix (using elementwise activation, here: ReLU)

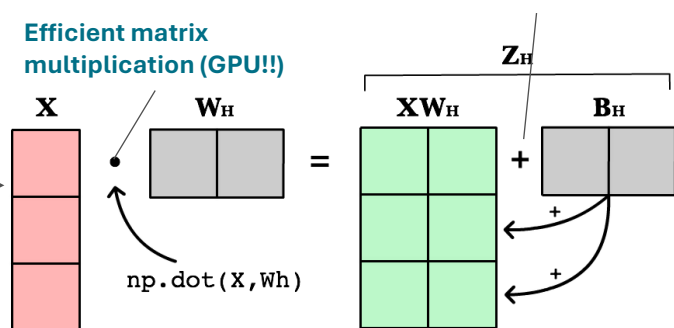
Z_O

... weighted net input for output layer



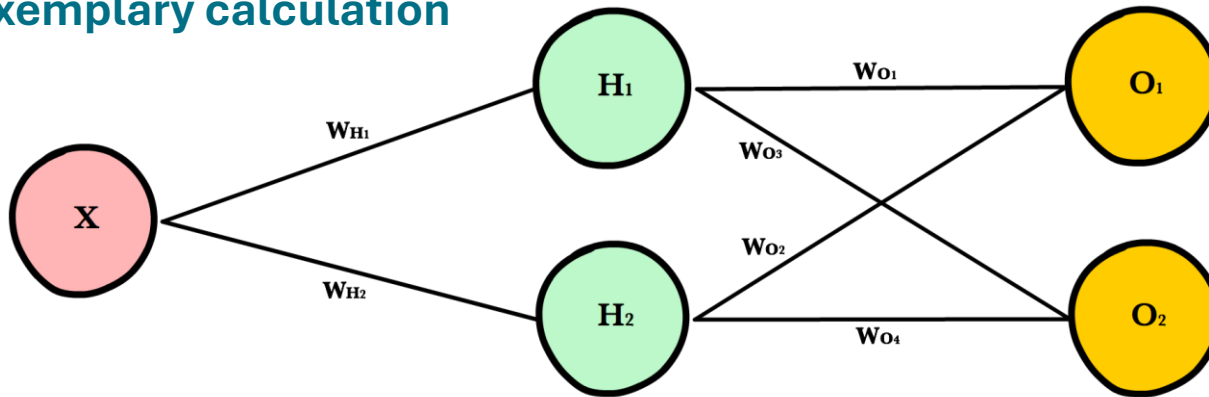
Efficient matrix multiplication (GPU!!)

Efficient matrix addition (GPU!!)



EXAMPLE: FORWARD PASS IN AN MLP

Exemplary calculation

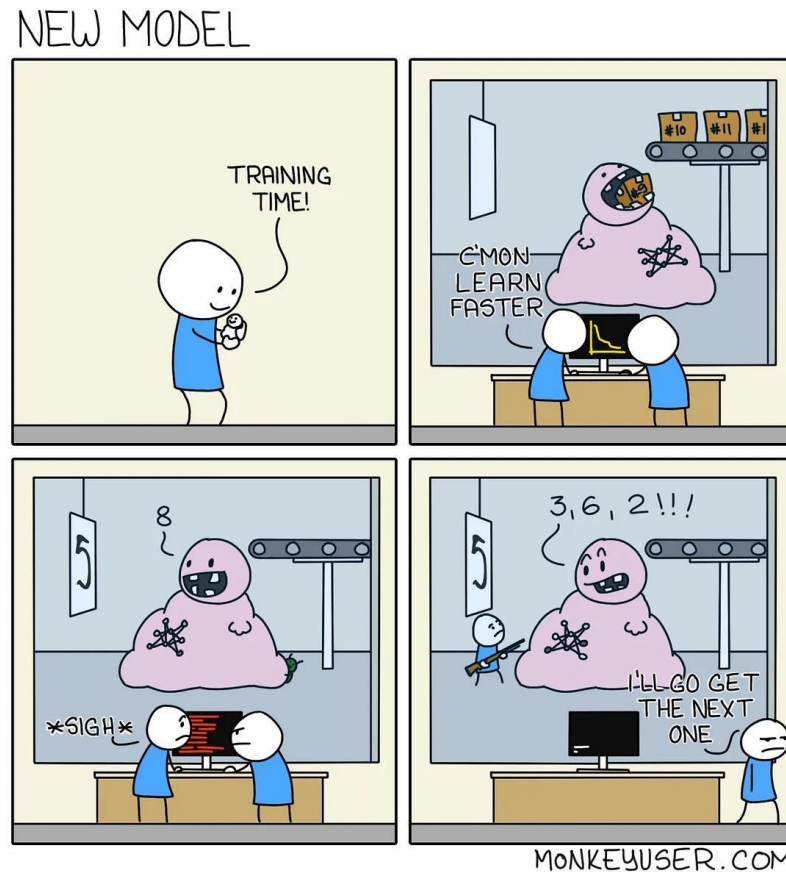


Task: Calculate the output of O_1 and O_2 applying the forward pass.

Given: $w_{H_1} = 0.2, w_{H_2} = 0.1$
 $H_1 = H_2 = O_1 = O_2 = f(x) = x$
 $w_{O_1} = 0.5, w_{O_2} = 0.3, w_{O_3} = 0.2, w_{O_4} = 0.7$
 $x = 0.8$
Activation function: Identity

NOW WHAT?

It is nice to know how we can process inputs with an MLP. **BUT – how to determine these weights?**



QUESTIONS?





DONE.

95028 Hof
Alfons-Goppel-Platz 1
Phone +49 9281 409-3000
Fax +49 9281 409-4000
mail@hof-university.de
www.hof-university.de